

BeachCamWeb

Training Pipeline — Technical Documentation

Beach Occupancy Prediction System

1. Overview

The BeachCamWeb pipeline is a fully automated end-to-end system that takes raw beach webcam images and produces deployed TFT (Temporal Fusion Transformer) forecast models for 3, 10, and 15-day beach occupancy predictions.

The pipeline is orchestrated by the Django management command `run_pipeline`, which chains 6 sequential steps. Steps can be run individually or from a specific point onward.

Pipeline Entry Point

```
python manage.py run_pipeline [OPTIONS]
```

Global Parameters

Parameter	Default	Description
<code>--step STEP</code>	<code>none</code>	Run a single step only (<code>crowd_count</code> , <code>export</code> , <code>build_dataset</code> , <code>train_data_notebook</code> , <code>train</code> , <code>evaluate</code>)
<code>--from-step STEP</code>	<code>none</code>	Skip all steps before STEP and run from there onward
<code>--since DATE</code>	<code>none</code>	ISO date passed to <code>export_snapshots</code> to filter snapshot export (e.g. 2024-01-01)
<code>--force-reprocess</code>	<code>false</code>	Force crowd counting to reprocess images already in cache
<code>--apply-crowd-count</code>	<code>false</code>	Include <code>crowd_count</code> step (excluded by default)
<code>--deploy-only</code>	<code>false</code>	Pass <code>--deploy-only</code> to the training script
<code>--light</code>	<code>false</code>	Light mode: 10% data, reduced sweeps. Use for testing pipeline flow

Quick Step Reference

1

crowd_count

Run Bayesian VGG-19 crowd counting on 2022 media images. Results cached as JSON per image.

2

export

Export Django snapshots and beach metadata to JSON files for dataset building.

3

build_dataset

Merge cache predictions + Django exports, fetch weather from Open-Meteo, and build a unified CSV dataset.

4

train_data_notebook

Run generate_train_data.ipynb via papermill to produce clean_datasets/ with filtered, analysis-ready CSVs.

5

train

Run TFT training script: feature selection + hyperparameter optimization for 3d / 10d / 15d models.

6

evaluate

Evaluate all deployed model sets via walk-forward cross-validation and write model_evaluation.json.

2. Step Details

Step 1 — crowd_count

Runs run_crowd_counting management command. Processes all cameras listed in beaches.json against images in PIPELINE_MEDIA_2022_DIR. Each image is passed to BayesianVGG19Predictor which wraps the VGG-19 Bayesian crowd counting model.

Results are stored in a flat JSON cache under PREDICTION_DIR/cache/predictions/bayesian_vgg19/. Cache keys are MD5 hashes of filename + lat/lon. Images already in cache are skipped unless --force-reprocess is set.

Parameter	Default	Description
--media-dir	settings.PIPELINE_MEDIA_2022_DIR	Path to the 2022 beach image archive (BeachCamDataset_NoVideos/)
--beaches-json	PREDICTION_DIR/beaches.json	JSON file mapping camera_key to lat/lon/name
--force-reprocess	false	Rerun predictions even if already in cache
--force-cpu	false	Use CPU instead of GPU (sets FORCE_CPU_BATCH=1)

Pipeline shortcut: only runs if --apply-crowd-count flag is passed to run_pipeline (disabled by default to avoid long re-runs).

Step 2 — export

Runs two management commands in sequence:

- export_snapshots — queries Django Snapshot model and serialises predictions, timestamps, webcam info, and beach metadata to django_export.json
- export_beach_profiles — exports WebCam static attributes (occupation grade, beach composition, proximity, etc.) to beach_profiles.json

Parameter	Default	Description
--since	none (all)	ISO date to filter snapshot export, e.g. 2024-01-01
output (export_snapshots)	EXPORTS_DIR/django_export.json	Output path for snapshot JSON
output (export_beach_profiles)	EXPORTS_DIR/beach_profiles.json	Output path for beach metadata JSON

Step 3 — build_dataset

Runs build_dataset_v4.py as a subprocess. Merges three data sources into a single CSV:

- Cache (2022-2023): prediction JSONs from crowd counting step
- Django (2025-2026): snapshots from the live system export
- Weather: fetched from Open-Meteo archive API per beach coordinate and timestamp

Beaches from cache and Django are matched by GPS coordinates (threshold 2 km by default). Matched beaches share a unique_id. The output CSV includes crowd counts, 21 weather variables (om_* prefix), and temporal features.

Parameter	Default	Description
--django	EXPORTS_DIR/django_export.json	Django snapshot export file(s)
--cache	CROWD_CACHE_DIR	Local crowd counting cache directory
--beach-profiles	EXPORTS_DIR/beach_profiles.json	Static beach metadata JSON
--output	PIPELINE_DIR/dataset.csv	Output CSV path
--threshold-km	2.0	Max distance in km for cache-Django beach matching

Step 4 — train_data_notebook

Executes generate_train_data.ipynb via papermill with the beachcam kernel. The notebook performs quality analysis and cleaning, then exports filtered datasets to clean_datasets_DDMMYY/.

After execution, the notebook is exported to HTML as a pipeline report, and clean_datasets/ is symlinked to the dated output directory.

Notebook parameters injected by papermill:

Parameter	Default	Description
CSV_PATH	dataset.csv	Filename of the raw dataset CSV (relative to PIPELINE_DIR)
OUT_DIR	clean_datasets_DDMMYY	Output directory name for clean CSVs

The notebook produces (inside OUT_DIR):

- `cache_2022_clean.csv` — filtered 2022 cache data
- `django_2025_clean.csv` — filtered 2025-2026 Django data
- `all_clean.csv` — union of both, used for training validation

Step 5 — train

Runs `tft_train_3_models.py` as a subprocess. Trains three separate TFT models (3d, 10d, 15d) with per-horizon optimization. Each model runs in a fresh subprocess to prevent CUDA memory corruption between sequential training jobs.

Training Phases (per horizon)

- Phase 1: Backward feature elimination on `hist_exog` weather features (stops at 3 features or when `relMAE` exceeds 2x baseline)
- Phase 3: Dropout sweep with fixed architecture (`hidden=64`, `heads=4`, `lr=0.001`)
- Phase 4: Deploy final model — saves `nf_model/`, `config.json`, `cv_predictions.csv`, `per_beach.csv`

Fixed Hyperparameters

Parameter	Default	Description
<code>hidden_size</code>	64	TFT hidden layer size
<code>n_head</code>	4	Number of attention heads
<code>lr</code>	0.001	Learning rate
<code>input_size</code>	48	Context window (4 days x 12h/day)
<code>batch_size</code>	32	Training batch size
<code>scaler_type</code>	minmax	Feature scaler
<code>loss</code>	MAE	Training loss function
<code>HOURS_PER_DAY</code>	12	Daytime hours filtered (8:00-20:00)

Script Parameters

Parameter	Default	Description
<code>--data-dir</code>	<code>clean_datasets</code>	Directory containing <code>cache_2022_clean.csv</code> and <code>django_2025_clean.csv</code>
<code>--prefix</code>	<code>tft_run</code>	Output directory prefix — timestamp appended automatically
<code>--cuda-device</code>	auto-detect	GPU device id (e.g. "0", "1"). Auto-selects GPU with most free VRAM
<code>--deploy-only</code>	false	Skip optimization; use known best config (<code>hidden=64</code> , <code>hist=cloud_low+shortwave+vpd</code>)
<code>--phase0</code>	false	Run optional horizon limit analysis (12h to 168h sweep) before main training
<code>--skip-eval</code>	false	Skip cross-validation evaluation after training
<code>--eval-dirs</code>	auto	Directories to evaluate on (default: <code>clean_datasets</code> + known backup dirs)
<code>--light</code>	false	Use 10% of series and reduced sweeps for pipeline testing

After training, `run_pipeline` automatically deploys the output models to `TFT_MODELS_DIR/<run_name>/tft_model_{3d,10d,15d}/` so they are discoverable by TFTService.

Step 6 — evaluate

Runs `evaluate_model_sets` management command. Iterates all model sets under `TFT_MODELS_DIR` and evaluates each on historical snapshot data using walk-forward cross-validation.

Outputs:

- `model_evaluation.json` — machine-readable metrics consumed by TFTService for feature reporting and model ranking in the UI
- `model_evaluation.csv` — human-readable summary

Metrics computed:

- `relMAE_all` — relative MAE over all months
- `relMAE_season` — relative MAE for Apr-Sep
- `relMAE_summer` — relative MAE for Jul-Aug

Parameter	Default	Description
<code>--sets</code>	<code>all</code>	Evaluate specific model set names only
<code>--since</code>	<code>auto</code>	Start date for evaluation window
<code>--until</code>	<code>auto</code>	End date for evaluation window
<code>--clear-cache</code>	<code>false</code>	Clear per-window error cache before running
<code>--clear-weather-cache</code>	<code>false</code>	Clear Open-Meteo weather cache before running

3. Global Pipeline Parameters

These flags are passed to `run_pipeline` and apply across multiple steps:

Parameter	Default	Description
<code>--step STEP</code>	<code>none</code>	Run a single step only (<code>crowd_count</code> , <code>export</code> , <code>build_dataset</code> , <code>train_data_notebook</code> , <code>train</code> , <code>evaluate</code>)
<code>--from-step STEP</code>	<code>none</code>	Skip all steps before STEP and run from there onward
<code>--since DATE</code>	<code>none</code>	ISO date passed to <code>export_snapshots</code> to filter snapshot export (e.g. 2024-01-01)
<code>--force-reprocess</code>	<code>false</code>	Force crowd counting to reprocess images already in cache
<code>--apply-crowd-count</code>	<code>false</code>	Include <code>crowd_count</code> step (excluded by default)
<code>--deploy-only</code>	<code>false</code>	Pass <code>--deploy-only</code> to the training script
<code>--light</code>	<code>false</code>	Light mode: 10% data, reduced sweeps. Use for testing pipeline flow

4. Key Files & Directories

Parameter	Default	Description
<code>settings.PIPELINE_MEDIA_2022_DIR</code>	<code>BeachCamDataset_NoVideos/</code>	2022 beach webcam image archive
<code>settings.PREDICTION_DIR</code>	<code>apps/prediction/</code>	Root for all pipeline artefacts
<code>settings.TFT_MODELS_DIR</code>	<code>apps/prediction/tft_models/</code>	Deployed model sets directory
<code>settings.TFT_EVAL_JSON</code>	<code>apps/prediction/model_evaluation.json</code>	Evaluation results (read by TFTService)
<code>settings.WEATHER_CACHE_DIR</code>	<code>apps/prediction/cache/weather/</code>	Open-Meteo archive cache
<code>settings.TFT_HINDCAST_CACHE_DIR</code>	<code>apps/prediction/cache/hindcast_cache/</code>	Hindcast prediction cache (filesystem JSON)
<code>PREDICTION_DIR/beaches.json</code>	<code>apps/prediction/beaches.json</code>	Camera key → lat/lon/name mapping for crowd counting
<code>PREDICTION_DIR/pipeline_workspace/</code>	<code>apps/prediction/pipeline_workspace/</code>	Working directory for training runs
<code>pipeline_workspace/clean_datasets/</code>	symlink	Symlink to latest <code>clean_datasets_DDMMYY/</code>
<code>pipeline_workspace/generate_train_data.ipynb</code>	—	Data cleaning and analysis notebook
<code>pipeline_workspace/generate_train_data_output.ipynb</code>	—	Executed notebook output
<code>pipeline_workspace/generate_train_data.html</code>	—	HTML report of last notebook run

5. Weather Module

`weather_module.py` provides weather data to both the dataset builder and the TFT service. It uses the Open-Meteo archive and forecast APIs exclusively (no AEMET — OM-only consistently outperforms the combined set).

APIs

- Archive API (`archive-api.open-meteo.com`) — fetches any historical date, cached as monthly JSON chunks per coordinate
- Forecast API (`api.open-meteo.com`) — fetches current + 16-day forecast, in-memory cache with 1-hour TTL

Cache Strategy

- Archive: monthly JSON files in `WEATHER_CACHE_DIR/archive/{lat}_{lon}_{Y}_{M}.json` — persistent, never expires
- Forecast: in-memory dict, 1-hour TTL, keyed by `(lat, lon, past_days, forecast_days)`
- Point results: individual JSON files in `WEATHER_CACHE_DIR/results/` — one file per `(lat, lon, date, hour)`

21 Weather Variables (om_ prefix)

temperature_2m, apparent_temperature, dewpoint_2m, relative_humidity_2m, pressure_msl, precipitation, rain, wind_speed_10m, wind_direction_10m, wind_gusts_10m, cloud_cover, cloud_cover_low, cloud_cover_mid, cloud_cover_high, visibility, uv_index, uv_index_clear_sky, sunshine_duration, evapotranspiration, vapour_pressure_deficit, direct_radiation, shortwave_radiation

8. Common Usage Examples

Full pipeline (everything except crowd counting)

```
python manage.py run_pipeline
```

Include crowd counting step

```
python manage.py run_pipeline --apply-crowd-count
```

Run from dataset building onward

```
python manage.py run_pipeline --from-step build_dataset
```

Run a single step

```
python manage.py run_pipeline --step train
```

Train with deploy-only (skip optimization, use known best config)

```
python manage.py run_pipeline --step train --deploy-only
```

Test pipeline flow with minimal data

```
python manage.py run_pipeline --from-step train --light
```

Re-export only recent snapshots and rebuild

```
python manage.py run_pipeline --from-step export --since 2025-06-01
```

Force reprocess all crowd counting images

```
python manage.py run_pipeline --step crowd_count --apply-crowd-count --force-reprocess
```

Evaluate specific model sets only

```
python manage.py evaluate_model_sets --sets tft_20260315  
tft_train_2022_validate_2025_summer
```